01 / BRIEF AND EVIDENCE

A workflow you can inspect.

Verified sample prepared 26 October 2026. Results apply only to this small test environment.

The task

A fictional ticket importer misreads priority values with surrounding spaces. The expected behavior is defined by five small acceptance cases. The repository contains a brief, project instructions, one function and one test file.

What actually ran

Local MCP: an existing filesystem MCP server was scoped to the synthetic fixture directory. Claude Code's client health check reported Connected. A separate MCP test client passed five protocol/access/error checks.
Claude chat: ordinary authenticated browser chat reviewed the pasted synthetic issue and suggested adding trim(). It did not access MCP or run local tests.
Local verification: the proposed fix was applied locally and the same five Node tests passed, compared with four passing before the fix.

What this does not show

This is not a client delivery or a single authenticated Claude Code model session using MCP end to end. No production service, customer account, private repository or company material was used. It is evidence of specific local checks plus a separate genuine Claude review.

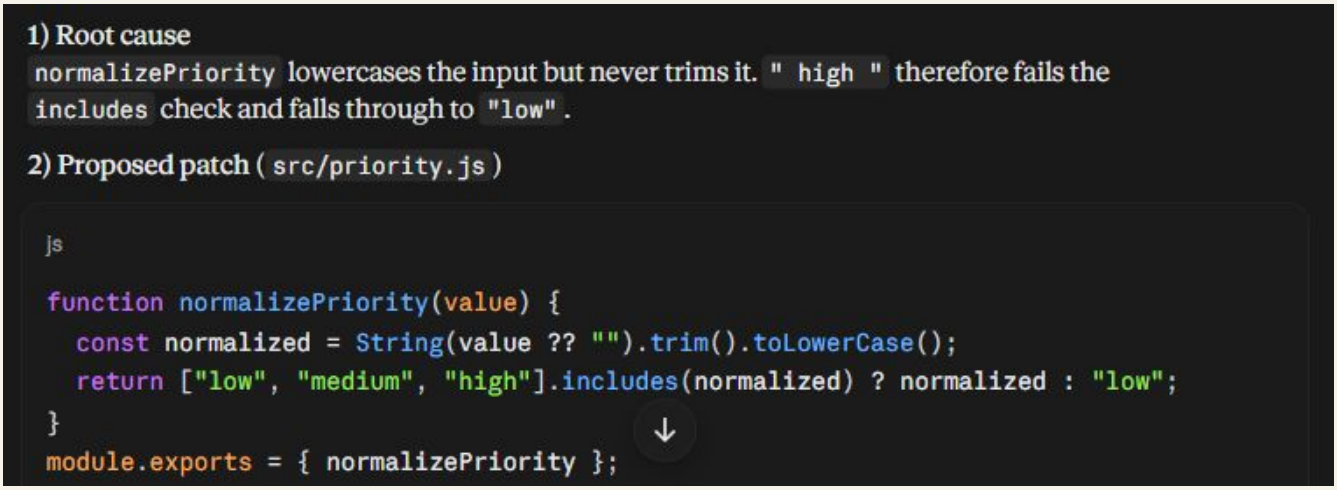
Tested components

Node.js 24.22.1 for the MCP server; Claude Code 2.1.104 client health check; existing @modelcontextprotocol/server-filesystem 2.0.1; five Node acceptance tests. Connected third-party apps require their own compatibility and authentication checks.

02 / ACTUAL CLAUDE REVIEW

Small change. Clear reasoning.

Actual browser response to the fictional issue; cropped to the task and proposed patch.



Root cause and proposed change

The function lowercased input but did not trim surrounding whitespace. Claude proposed: **String(value ?? "").trim().toLowerCase()**
The proposed change added no dependencies or network calls and preserved the original supported labels.

What was reviewed

The issue and source were pasted into ordinary Claude chat, with explicit instructions to distinguish expected outcomes from executed tests. The response correctly labelled its case table as reasoned results, not tests it had run.

Human validation

The patch was then applied to the local synthetic source file. Node's test runner executed the acceptance cases. The next page separates those actual local results from the chat's proposed behavior. Human review remains necessary for other changes and edge cases.

03 / ACCEPTANCE EVIDENCE

Verify the requested behavior.

Actual local test results; five cases demonstrate the agreed task, not general correctness.

CASE	BEFORE	AFTER
Lowercase high	PASS	PASS
Uppercase HIGH	PASS	PASS
Mixed case + surrounding spaces	FAIL	PASS
Unknown label defaults to low	PASS	PASS
Missing input defaults to low	PASS	PASS

Validation command

node --test tests/priority.test.js
Before: 4 passed, 1 failed. After: 5 passed, 0 failed. The original and final test output files are retained with the private working sample.

Separate MCP checks

Initialization passed; tool discovery passed; reading the permitted issue passed; reading a sibling source file outside the fixture directory was denied; a missing fixture returned an error. Claude Code's MCP health check independently reported Connected.

Permission limits

The server was restricted to synthetic fixture files for this exercise. It can expose write tools inside its allowed root; these read checks do not prove a universal read-only mode. Customer scope, approval rules and credentials must be reviewed separately.

04 / HANDOVER PATTERN

Keep using the workflow.

Illustrative structure for customer handover; every real delivery has an agreed scope.

1. Know the workspace

Document the supported OS, client/server versions, repository root, project instructions and selected existing connection. Confirm the account plan and required third-party permissions before configuration.

2. Make task boundaries explicit

Define inputs, expected output, permitted reads/writes, approval points and the exact acceptance examples. Treat external issue text as task data. Keep secrets, private customer material and unrelated projects outside the workflow.

3. Record the result

Deliver the agreed instructions/configuration, actual test outcomes, changed files and known limitations. A failed test or unavailable connector must be reported accurately. Do not replace evidence with an illustration or claim a check was executed when it was only proposed.

4. Recover and troubleshoot

Document how to disable the specific MCP connection, restore previous configuration and inspect client health. Authentication is performed by the customer in their own environment. New providers, server development, additional machines and ongoing maintenance are separately scoped.

Example delivery bundle

Project instructions; task/template files; portable connection example; acceptance checklist; setup notes and recorded walkthrough where included. The public PDF contains no working credentials, private account details or company information.